

26 Fase 1 - Analisador Léxico e Gerador de Assembly para ARMv7

Este trabalho pode ser realizado em grupos de até 4 alunos. **Grupos com mais de 4 membros terão o trabalho anulado.** Leia todo este texto antes de começar e siga o seguinte código de ética: *você pode discutir as questões com colegas, professores e amigos, consultar livros da disciplina, bibliotecas virtuais ou físicas, e a Internet em geral, em qualquer idioma. Você pode usar qualquer ferramenta de Inteligência Artificial para dúvidas, mas não para fazer o trabalho no seu lugar. O trabalho é seu e deve ser realizado por você. Trabalhos com indicativo de realização por inteligência artificial, ou com indicativo de cópia serão anulados.*

Os trabalhos entregues serão avaliados por uma ferramenta de Inteligência Artificial, que verificará a originalidade do texto e a autoria do código. Em seguida o trabalho será avaliado pelo professor e finalmente o trabalho deverá ser defendido por um dos alunos do grupo em uma seção de prova de autoria conforme as regras disponíveis no plano de ensino.

O trabalho deve ser entregue por meio da URL de um repositório público no **Github** contendo o código-fonte, arquivos de teste, documentação e código Assembly gerado. O repositório deve ser organizado com *commits* claros e as contribuições de cada aluno devem estar registradas na forma de *pull requests*.

26.1 Objetivo

Pesquisar e praticar conceitos de analisador léxico para desenvolver um programa em **Python, C, ou C++** que processe expressões aritméticas em notação polonesa reversa (RPN), conforme definida neste texto, a partir de um arquivo de texto, utilizando **máquinas de estado finito** (FSMs) implementadas obrigatoriamente com funções. O programa deve executar as expressões em um ambiente de teste (ex.: o notebook do aluno, ou um computador da instituição) usando como ambiente de execução o **simulador disponível em [CPULATOR](#) usando obrigatoriamente modelo ARMv7 DEC1-SOC(v16.1).**

O seu trabalho será criar um analisador léxico e, a partir do *string de tokens* gerar um código Assembly, compatível com a arquitetura ARMv7 DEC1-SOC(v16.1) que represente o programa de testes.

26.2 Descrição do Trabalho

Seu objetivo é desenvolver um programa (em Python, C ou C++) capaz de:

1. Ler um arquivo de texto contendo expressões aritméticas em **Escritas RPN**, segundo o formato especificado neste documento, com uma expressão por linha. Este arquivo contém o código do programa que será analisado pelo analisador léxico.
2. Analisar as expressões usando um analisador léxico baseado em Autômatos Finitos Determinísticos, com estados implementados por funções.
3. Transformar as expressões em um texto contendo o código Assembly para o Cpulador-ARMv7 DEC1-SOC(v16.1). As operações descritas no texto de entrada serão realizadas no Cpulador-ARMv7 DEC1-SOC(v16.1).
4. Garantir que o resultado do programa, ou as interações necessárias, sejam realizadas por meio das interfaces disponíveis no Cpulador-ARMv7 DEC1-SOC(v16.1) (display, leds, botões, chaves, etc.).
5. Hospedar o código, arquivos de teste e documentação em um repositório público no **GitHub**.

26.2.1 Características Especiais da Linguagem

As expressões devem ser escritas em notação RPN, no formato **(A B op)**, no qual **A** e **B** são **números reais de 64 bits**, e **op** é um operador aritmético entre os listados neste documento. O programa deve suportar apenas as operações aritméticas básicas listadas neste documento e os comandos especiais para manipulação de memória também listados neste documento. Além disso, o programa deve ser capaz de lidar com expressões aninhadas sem limites de aninhadas.

Para a criação da nossa linguagem de programação, considere a seguinte sintaxe:

- Considerando que **A** e **B** são números reais, e usando o ponto como separador decimal, (ex.: 3.14), teremos:

1. Operadores suportados na Fase 1:

- Adição: **+** (ex.: **(A B +)**);
- Subtração: **-** (ex.: **(A B -)**);
- Multiplicação: ***** (ex.: **(A B *)**);
- Divisão real: **/** (ex.: **(A B /)**);
- Divisão inteira: **/** (ex.: **(A B //)** para inteiros);
- Resto da divisão inteira: **%** (ex.: **(A B %)**);

- Potenciação: $^$ (ex.: $(A \ B \ ^)$, onde B é um inteiro positivo A^B);
2. Todas as operações (exceto divisão inteira e resto) usam números reais codificados em 64 bits segundo a **norma IEEE 754**. A página [Os desafios da norma IEEE 754](#) contém informações relevantes sobre a norma **IEEE 754** para a realização desta tarefa.
 3. Expressões podem ser aninhadas sem limite, por exemplo:
 - $(A \ (C \ D \ *) \ +)$: Soma A ao produto de C e D ;
 - $((A \ B \ *) \ (D \ E \ *) \ /)$: Divide o produto de A e B pelo produto de D e E .
 - $((A \ B \ +) \ (C \ D \ *) \ /)$: Divide a soma de A e B pelo produto de C e D .

Warning

A ordem de precedência das operações segue a ordem de precedência usual em matemática.

26.2.2 Comandos Especiais

A linguagem que estamos criando inclui três comandos especiais para manipulação de memória e resultados:

1. $(N \ RES)$: Retorna o resultado da expressão N linhas anteriores (N é um inteiro não negativo).
2. $(V \ MEM)$: Armazena o valor real V em uma memória chamada MEM .
3. (MEM) : Retorna o valor armazenado em MEM . Se a memória não foi inicializada, retorna 0.0.

Nos quais: - MEM pode ser qualquer conjunto de letras maiúsculas, tal como MEM , VAR , X , etc. - RES é uma *keyword* da linguagem que estamos criando. A única *keyword* da linguagem nesta fase.

Warning

Cada arquivo de texto, código fonte da linguagem que estamos criando, representa um escopo independente de memória.

26.3 Analisador Léxico com Autômatos Finitos Determinístico

1. O analisador léxico deve ser implementado usando Autômatos Finitos Determinísticos, com **cada estado representado por uma função**. Qualquer forma diferente de implementação provocará o zeramento do trabalho.

2. O Autômato Finito Determinístico deve reconhecer *tokens* válidos na linguagem: números reais, operadores (+, -, *, ^, %, ^), e os comandos especiais (RES, MEM). Além, é claro dos sinais de abertura e fechamento de parênteses.
3. Funções de teste específicas devem ser criadas para validar o analisador léxico, cobrindo:
 - Entradas válidas (ex.: (3.14 2.0 +), (5 RES), (10.5 CONTADOR));
 - Entradas inválidas (ex.: (3.14 2.0 &), números malformados como 3.14.5, 3,45 ou parênteses desbalanceados).

Warning

O uso de expressões regulares, ou das bibliotecas de expressões regulares disponíveis em Python, C ou C++, para a implementação do analisador léxico, é proibido e resultará no zeramento do trabalho.

26.4 Arquivos de Teste

Para que seu trabalho possa ser avaliado você deverá: 1. Fornecer o **mínimo de 3 arquivos de texto**, cada um com **pelo menos 10 linhas** de expressões aritméticas escritas segundo a linguagem definida neste documento. 2. Cada arquivo deve incluir todas as operações (+, -, *, /, %, ^) e comandos especiais (N RES), (V MEM), (MEM). 3. Os arquivos devem estar no mesmo diretório do código-fonte e ser processados via argumento de linha de comando (ex.: ./NomeDoSeuPrograma teste1.txt). 4. O programa não deve incluir menu ou qualquer seleção interativa de arquivos.

26.5 Hospedagem no GitHub

O projeto deve ser hospedado em um repositório público no **GitHub**. O repositório deve ser criado por um dos alunos do grupo e deve ter o mesmo nome grupo que aparece no ambiente virtual da instituição onde o grupo está registrado. O repositório deve conter: 1. Código-fonte do programa; 2. Arquivos de teste (mínimo 3); 3. Funções de teste para o analisador léxico; 4. Última versão do Código Assembly para CpuLator-ARMv7 DEC1-SOC(v16.1) gerado pela última execução do seu projeto; 5. Documentação (ex.: arquivo README.md) explicando como compilar, executar e testar o programa. Contendo, no mínimo, o nome da instituição, disciplina, professor e nome dos alunos do grupo, em ordem alfabética seguido do usuário deste aluno no **GitHub**. 6. O repositório deve ser organizado com *commits* claros, as contribuições de cada um dos alunos devem estar registradas na forma de *pull requests*.

26.6 Requisitos do Código

1. As primeiras linhas do código devem conter:
 - Nomes dos integrantes do grupo, em ordem alfabética seguidos do usuário deste aluno no **GitHub**.
 - Nome do grupo no ambiente virtual de aprendizagem (Canvas).
2. O programa deve receber o nome do arquivo de teste como argumento na linha de comando. Nenhum menu será necessário.
3. O código deve ser escrito em **Python**, **C**, ou **C++**. Com as funções nomeadas como está explicitado na [Section 26.7](#).
4. A última versão do código Assembly gerado deve ser funcional e incluído no repositório. A versão em Assembly deve ser exatamente o mesmo algoritmo explicitado no último arquivo de testes executado de forma que o resultados do programa seja obtido pelo cálculo realizado Cpulador-ARMv7 DEC1-SOC(v16.1).

26.7 Divisão de Tarefas para a Fase 1

Para resolver o problema de processamento da expressões definidas nos arquivos de teste da fase 1, o trabalho será dividido entre até quatro alunos, trabalhando independentemente, na mesma sala, ou de forma remota. Cada aluno será responsável por uma parte específica do sistema, com interfaces claras para facilitar a integração. Abaixo está uma **sugestão da divisão das tarefas**, considerando as funções solicitadas: `parseExpressao`, `executarExpressao`, `gerarAssembly`, e `exibirResultados`.

Warning

As tarefas podem ser divididas da forma que cada grupo achar mais conveniente, desde que as funções e interfaces sejam respeitadas. **Nota:** a divisão é sugestão, o nome das funções é obrigatório.

Warning

Nota: O vetor de *tokens* gerado pelo Analisador Léxico deve ser salvo em um arquivo .txt para uso nas próximas fases do projeto. Cabe ao grupo decidir o formato que será usado para salvar os *tokens* (csv, json, etc.). Apenas os *tokens* referentes a última execução do código do analisador léxico devem estar salvos no repositório do **GitHub**.

26.7.1 Aluno 1: Função `parseExpressao` e Analisador Léxico com Autômato Finito Determinístico

Responsabilidades: Criar e administrar o repositório no **GitHub**. Além disso:

1. Implementar `parseExpressao(std::string linha, std::vector<std::string>& _tokens_)` (ou equivalente em Python/C) para analisar uma linha de expressão RPN e extrair *tokens*.
2. Implementar o analisador léxico usando Autômatos Finitos Determinísticos (AFDs), com cada estado como uma função (ex.: `estadoNumero`, `estadoOperador`, `estadoParenteses`).
3. Validar *tokens*:
 - Números reais (ex.: `3.14`) usando ponto como separador decimal;
 - Operadores (`+`, `-`, `*`, `/`, `%`, `^`);
 - Comandos especiais (`RES`, `MEM`) e parênteses;
 - Detectar erros como números malformados (ex.: `3.14.5`), parênteses desbalanceados ou operadores inválidos;
 - Criar funções de teste para o analisador léxico, cobrindo entradas válidas e inválidas
4. Escrever `parseExpressao` para dividir a linha em *tokens* usando um Autômato Finito Determinístico;

Interface:

- Recebe uma linha de texto e retorna um vetor de *tokens*;
- Fornece *tokens* válidos para `executarExpressao`.

26.7.2 Aluno 2: Função `executarExpressao` e Gerenciamento de Memória

Responsabilidades:

1. Testar o Autômato Finito Determinístico com entradas diversificadas (`3.14 2.0 +`), (`5 RES`), (`3.14.5 2.0 +`) (inválido);
2. Criar um método, antes do Autômato Finito Determinístico, para lidar com parênteses aninhados.
3. Implementar `executarExpressao(...)` usando uma estrutura de dicionário/mapa para gerenciar múltiplas variáveis na memória.;
4. Gerenciar a memória `MEM` para comandos (`V MEM`) e (`MEM`);
5. Manter um histórico de resultados para suportar (`N RES`);
6. Criar funções de teste para validar a execução de expressões e comandos especiais.

Tarefas Específicas:

- Usar uma pilha para avaliar expressões RPN (ex.: em **C++**: `std::stack<float>`);
- Implementar operações (+, -, *, /, %, ^) com precisão de 64 bits (IEEE 754);
- Tratar divisão inteira e resto separadamente;

Interface:

- Recebe *tokens* de `parseExpressao` e atualiza `resultados` e `memoria`;
- Fornece resultados para `exibirResultados` e Assembly.

26.7.3 Aluno 3: Função `gerarAssembly` e Leitura de Arquivo**Responsabilidades:**

1. Testar com expressões como `(3.14 2.0 +)`, `((1.5 2.0 *) (3.0 4.0 *) /)`, `(5.0 MEM)`, `(2 RES)`;
2. Verificar erros como divisão por zero ou `N` inválido em `(N RES)`.
3. Implementar `gerarAssembly(const std::vector<std::string>& _tokens_, std::string& codigoAssembly)` para gerar o código Assembly;
4. Implementar `lerArquivo(std::string nomeArquivo, std::vector<std::string>& linhas)` para ler o arquivo de entrada;
5. Criar funções de teste para validar a leitura de arquivos e a geração de Assembly; Lembre-se o Assembly deve conter todas as operações do texto de teste.

Tarefas Específicas:

- A função `gerarAssembly` deve receber o vetor de tokens gerado pelo analisador léxico e traduzi-lo para Assembly ARMv7.
- Gerar Assembly ARMv7 para operações RPN e comandos especiais;
- Testar com arquivos contendo 10 linhas, ou mais, incluindo expressões aninhadas e comandos especiais;
- Verificar erros de abertura de arquivo e exibir mensagens claras.

Interface:

- `lerArquivo` fornece linhas para `parseExpressao`;
- `gerarAssembly` produz código Assembly.

26.7.4 Aluno 4: Função `exibirResultados`, Interface do Usuário e Testes

Responsabilidades:

1. Alertar se o arquivo tiver linhas malformadas ou exceder limites.
2. Implementar `exibirResultados(const std::vector<float>& resultados)` para exibir os resultados das expressões;
3. Implementar e gerenciar a interface no `main`, incluindo leitura do argumento de linha de comando;
4. Corrigir problemas de entrada (ex.: em **C++**: `std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');`);
5. Criar funções de teste para validar a saída e o comportamento do programa completo.

Tarefas Específicas:

- Exibir resultados com formato claro (ex.: uma casa decimal para números reais);
- Implementar o `main` para chamar `lerArquivo`, `parseExpressao`, `executarExpressao`, e `exibirResultados`;
- Testar com arquivos de teste fornecidos, verificando saídas para expressões simples e complexas;
- Testar o FSM com comandos especiais como `(V MEM)` e `(MEM)`;

Interface:

- Usa resultados de `executarExpressao` para exibir saídas;
- Gerencia a execução do programa via argumento de linha de comando.

26.8 Considerações para Integração

- **Interfaces:** concordar com assinaturas das funções e formatos de dados (ex.: vetor de *tokens*, resultados em binário 64 bits);
- **Depuração:** testar cada parte isoladamente, simulando entradas/saídas;
- **Passos de Integração:**
 1. Copiar `main` do Aluno 4;
 2. Inserir `lerArquivo` e `gerarAssembly` do Aluno 3;

3. Adicionar `executarExpressao` do Aluno 2;
 4. Incluir `parseExpressao` do Aluno 1;
- **Resolução de Conflitos:** discutir problemas imediatamente na sala, ou de forma remota. Idealmente, todos os problemas, e todo o processo de desenvolvimento deve estar registrado com as ferramentas disponíveis no **GitHub** (ex.: `issues`, `pull requests`, `commits`).
 - **Depuração Final:** Testar o programa com os 3 arquivos de teste, verificando expressões, comandos especiais, e saída Assembly.

Warning

Você pode fazer um programa em paralelo, que não será incluído no repositório do seu projeto, para testar as funções que criar. Neste programa de comparação, você pode usar os recursos que desejar.

26.9 Avaliação

O trabalho será avaliado antes da prova de autoria, primeiro por um sistema de inteligência artificial e depois pelo professor, com os seguintes critérios:

1. Cálculos e Funcionalidades (70%):

1. Implementação completa de todas as operações RPN e comandos especiais.
 - Cada operação não implementada reduz 10% dos 70% deste item.
 - Falha na divisão inteira reduz 50% dos 70% deste item.
2. Analisador léxico com Autômato Finito funcional e testado.
 - Uso de ponto flutuante com precisão inferior a 64 bits, ou diferente da norma IEEE754 reduz 90% dos 70% deste item.

2. Organização e Legibilidade do Código (15%):

- Código claro, comentado e bem estruturado.
- Repositório GitHub organizado, com README completo, explicando o projeto e atendendo as condições definidas anteriormente.

3. Robustez (15%):

- Tratamento de erros em expressões complexas e entradas inválidas.

- Testes do analisador léxico cobrindo todos os casos.

26.10 Prova de Autoria

- Um aluno do grupo será sorteado, por um sistema disponível *online* (<https://frankalcantara.com/sorteio>) para responder uma pergunta em uma lista de até 10 perguntas;
- Falha na explicação do trabalho, ou na resposta reduz 35% da nota obtida na Avaliação do Projeto.

Warning

O aluno sorteado, e apenas o aluno sorteado poderá contestar a avaliação feita pelo professor antes da prova de autoria.

26.11 Entrega

- O repositório GitHub deve conter:
 - Código-fonte (Python, C, ou **C++**);
 - Três arquivos de teste com expressões RPN;
 - Funções de teste para o analisador léxico;
 - Código Assembly da última execução do analisador léxico;
 - Arquivo de texto contendo os *tokens* gerados na última execução do analisador léxico;
 - README com instruções de compilação, execução e testes;
- O programa deve ser executado com o comando `./NomeDoSeuPrograma teste1.txt`. Ou o equivalente a isso em Python.

Copyright © 2025,2026 Frank de
Alcantara